12.4 认证与授权

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~200,000 tokens，输出 ~15,000 tokens（本章合计）

Gateway 是 OpenClaw 系统的入口大门。任何客户端——无论是本地的 CLI、远程的 macOS 应用、还是浏览器中的 Web 控制台——都需要通过 Gateway 的认证才能访问系统。本节分析 OpenClaw 的多层认证机制。

12.4.1 Gateway Token 认证

Token 认证是最基础的认证方式。原理很简单：Gateway 启动时配置一个密钥（Token），客户端连接时提供同样的密钥，匹配则通过。

认证配置解析

`src/gateway/auth.ts` 中的 `resolveGatewayAuth()` 函数负责解析认证配置：

```typescript
// src/gateway/auth.ts
export type ResolvedGatewayAuth = {
  mode: "token" | "password";  // 认证模式
  token?: string;              // Token 值
  password?: string;           // 密码值
  allowTailscale: boolean;     // 是否允许 Tailscale 认证
};

export function resolveGatewayAuth(params: {
  authConfig?: GatewayAuthConfig | null;
  env?: NodeJS.ProcessEnv;
  tailscaleMode?: GatewayTailscaleMode;
}): ResolvedGatewayAuth {
  const authConfig = params.authConfig ?? {};
  const env = params.env ?? process.env;

  // Token 的来源优先级：配置文件 > OPENCLAW 环境变量 > CLAWDBOT 环境变量
  const token =
    authConfig.token ??
    env.OPENCLAW_GATEWAY_TOKEN ??
    env.CLAWDBOT_GATEWAY_TOKEN ??
    undefined;

  // 密码的来源优先级：同上
  const password =
    authConfig.password ??
    env.OPENCLAW_GATEWAY_PASSWORD ??
    env.CLAWDBOT_GATEWAY_PASSWORD ??
    undefined;

  // 模式推断：如果配置了 password 但没指定 mode，默认用 password 模式
  const mode = authConfig.mode ?? (password ? "password" : "token");

  return { mode, token, password, allowTailscale };
}
```

注意 `CLAWDBOT_GATEWAY_TOKEN` 等环境变量——这是对旧版本（项目曾用名 ClawdBot）的向后兼容。在大型项目中，这种对旧配置名的兼容是非常常见的实践。

配置校验

启动时， `assertGatewayAuthConfigured()` 确保认证已正确配置：

```
export function assertGatewayAuthConfigured(auth: ResolvedGatewayAuth): void {
  if (auth.mode === "token" && !auth.token) {
    if (auth.allowTailscale) return; // Tailscale 模式可以不需要 Token
    throw new Error(
      "gateway auth mode is token, but no token was configured " +
      "(set gateway.auth.token or OPENCLAW_GATEWAY_TOKEN)"
    );
  }
  if (auth.mode === "password" && !auth.password) {
    throw new Error("gateway auth mode is password, but no password was configured");
  }
}
```

如果 Token 没有配置且不允许 Tailscale 认证，Gateway 会拒绝启动并给出清晰的错误提示。这是一种 **Fail-Fast**（快速失败）设计——宁可在启动时就发现问题，也不要在运行时出现安全漏洞。

连接认证流程

当客户端发起 WebSocket 连接并发送 `connect` 消息后，Gateway 调用 `authorizeGatewayConnect()` 进行认证：

```typescript
export async function authorizeGatewayConnect(params: {
  auth: ResolvedGatewayAuth;
  connectAuth?: ConnectAuth | null;
  req?: IncomingMessage;
  trustedProxies?: string[];
  tailscaleWhois?: TailscaleWhoisLookup;
}): Promise<GatewayAuthResult> {
  const { auth, connectAuth, req, trustedProxies } = params;
  const localDirect = isLocalDirectRequest(req, trustedProxies);

  // 1. 尝试 Tailscale 认证（仅对非本地请求）
  if (auth.allowTailscale && !localDirect) {
    const tailscaleCheck = await resolveVerifiedTailscaleUser({ req, ... });
    if (tailscaleCheck.ok) {
      return { ok: true, method: "tailscale", user: tailscaleCheck.user.login };
    }
  }

  // 2. Token 认证
  if (auth.mode === "token") {
    if (!auth.token)         return { ok: false, reason: "token_missing_config" };
    if (!connectAuth?.token)  return { ok: false, reason: "token_missing" };
    if (!safeEqual(connectAuth.token, auth.token))
      return { ok: false, reason: "token_mismatch" };
    return { ok: true, method: "token" };
  }

  // 3. 密码认证
  if (auth.mode === "password") {
    if (!auth.password)       return { ok: false, reason: "password_missing_config" };
    if (!connectAuth?.password) return { ok: false, reason: "password_missing" };
    if (!safeEqual(password, auth.password))
      return { ok: false, reason: "password_mismatch" };
    return { ok: true, method: "password" };
  }

  return { ok: false, reason: "unauthorized" };
}
```

认证结果 `GatewayAuthResult` 包含认证方法信息：

```typescript
export type GatewayAuthResult = {
  ok: boolean;
  method?: "token" | "password" | "tailscale" | "device-token";
  user?: string;     // 对于 Tailscale 认证，记录用户名
  reason?: string;   // 失败原因
};
```

时间安全比较

Token 比较使用了 `safeEqual()` 函数而非简单的 `===`：

```typescript
function safeEqual(a: string, b: string): boolean {
  if (a.length !== b.length) return false;
  return timingSafeEqual(Buffer.from(a), Buffer.from(b));
}
```

> **衍生解释**：`timingSafeEqual`（时间安全比较）是一种防止**时序攻击**（Timing Attack）的技术。普通的字符串比较 `===` 在发现第一个不匹配的字符时就会返回 `false`，这意味着比较时间与"匹配了多少字符"成正比。攻击者可以通过精确测量比较时间来逐字符猜测密钥。
>
> `timingSafeEqual` 无论字符串内容如何，都会花费**恒定的时间**完成比较，从而消除了时序侧信道。这是 Node.js `crypto` 模块提供的标准安全原语。

本地请求检测

`isLocalDirectRequest()` 函数用于判断请求是否来自本机：

```
export function isLocalDirectRequest(
  req?: IncomingMessage,
  trustedProxies?: string[]
): boolean {
  const clientIp = resolveRequestClientIp(req, trustedProxies) ?? "";
  if (!isLoopbackAddress(clientIp)) return false;

  const host = getHostName(req.headers?.host);
  const hostIsLocal = host === "localhost" || host === "127.0.0.1" || host === "::1";
  const hostIsTailscaleServe = host.endsWith(".ts.net");

  const hasForwarded = Boolean(
    req.headers?.["x-forwarded-for"] ||
    req.headers?.["x-real-ip"] ||
    req.headers?.["x-forwarded-host"]
  );

  const remoteIsTrustedProxy = isTrustedProxyAddress(
    req.socket?.remoteAddress, trustedProxies
  );
  return (hostIsLocal || hostIsTailscaleServe) && (!hasForwarded || remoteIsTrustedProx
}
```

这个函数的逻辑比看起来复杂，因为它需要防范**伪造本地请求**的攻击：

1. **IP 检查**：客户端 IP 必须是回环地址（`127.0.0.1`、`::1` 等）
2. **Host 头检查**：请求的 Host 头必须指向本地（`localhost`、`127.0.0.1`）或 Tailscale 地址
3. **代理检测**：如果存在 `X-Forwarded-For` 等代理头，只有当来源 IP 是受信代理时才认为是本地请求

为什么这么复杂？考虑以下攻击场景：恶意网页通过 JavaScript 向 `http://localhost:18789` 发起请求（跨站请求），如果 Gateway 只检查 IP 而不检查 Host 头和代理头，就可能被欺骗认为是"本地请求"并跳过认证。

12.4.2 设备配对（Device Pairing）机制

Token 认证适合受信环境（CLI、服务器端），但对于移动设备（iOS/Android），直接在设备上配置 Token 既不安全也不方便。OpenClaw 引入了**设备配对**（Device Pairing）机制来解决这个问题。

> **衍生解释**：设备配对是一种在两个设备之间建立信任关系的过程，类似于蓝牙配对。核心思想是通过一个**带外信道**（Out-of-Band Channel）——如显示在屏幕上的配对码——来验证双方身份，然后交换密钥建立持久的信任关系。

配对流程

设备配对基于**非对称密钥加密**（Public Key Cryptography）：

```
┌──────────────────────────────────────────────────────────────┐
│ 第 1 步：设备生成密钥对                                        │
│                                                              │
│ iPhone ──→ 生成 Ed25519 密钥对 {publicKey, privateKey}        │
│            保存 privateKey 到 Keychain                         │
└──────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────┐
│ 第 2 步：发起配对请求                                          │
│                                                              │
│ iPhone ──WebSocket──→ Gateway                                │
│   发送 device.pair.request:                                   │
│   { deviceId, publicKey, clientId, role, scopes }            │
│                                                              │
│ Gateway ──event──→ 已认证的管理客户端（如 macOS 应用、CLI）      │
│   推送 device.pair.requested 事件:                            │
│   { requestId, deviceId, publicKey, platform, remoteIp }     │
└──────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────┐
│ 第 3 步：管理员审批                                            │
│                                                              │
│ macOS 应用显示配对请求:                                        │
│   "iPhone (192.168.1.100) 请求连接，是否批准？"                 │
│                                                              │
│ 用户点击 "批准" ──→ device.pair.approve({ requestId })        │
│ Gateway 生成设备令牌，返回给 iPhone                            │
└──────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────┐
│ 第 4 步：后续连接使用签名认证                                   │
│                                                              │
│ iPhone ──→ 构建认证载荷 ──→ 用 privateKey 签名 ──→ 发送        │
│ Gateway ──→ 用存储的 publicKey 验证签名 ──→ 认证通过            │
└──────────────────────────────────────────────────────────────┘
```

签名载荷构建

设备在后续连接时需要对一段标准化的载荷进行签名。 `buildDeviceAuthPayload()` 函数定义了载荷格式：

```
// src/gateway/device-auth.ts
export function buildDeviceAuthPayload(params: DeviceAuthPayloadParams): string {
  const version = params.version ?? (params.nonce ? "v2" : "v1");
  const scopes = params.scopes.join(",");
  const token = params.token ?? "";
  const base = [
    version,            // 载荷版本
    params.deviceId,    // 设备 ID
    params.clientId,    // 客户端类型
    params.clientMode,  // 客户端模式
    params.role,        // 角色
    scopes,             // 权限范围
    String(params.signedAtMs), // 签名时间戳
    token,              // 设备令牌
  ];
  if (version === "v2") {
    base.push(params.nonce ?? ""); // v2 版本增加随机数防重放
  }
  return base.join("|");
}
```

载荷使用管道符 `|` 分隔各字段，类似于 JWT 的载荷格式但更简单。注意几个安全设计：

1. **签名时间戳**（`signedAtMs`）：防止签名被无限期重用。服务器可以拒绝时间戳过旧的签名。
2. **版本演进**：v2 版本增加了 `nonce`（随机数），用于防止**重放攻击**——即使攻击者截获了一个有效的签名，也无法重新使用它，因为每次连接的 nonce 不同。

> **衍生解释**：重放攻击（Replay Attack）是指攻击者截获一个合法的认证消息后，重新发送它来获得未授权的访问。这在网络安全中是一类经典攻击。对策包括时间戳（签名过期）、随机数（每次不同）、以及序列号（检测重复）。

连接参数中的设备认证

回顾 `ConnectParams` 中的 `device` 字段：

```
device: Type.Optional(Type.Object({
  id: NonEmptyString,          // 设备 ID
  publicKey: NonEmptyString,   // 公钥
  signature: NonEmptyString,   // 签名
  signedAt: Type.Integer(),    // 签名时间戳
  nonce: Type.Optional(NonEmptyString), // 随机数（v2）
}));
```

Gateway 收到这些信息后：

1. 在已配对设备列表中查找 `device.id`
2. 使用存储的公钥验证 `signature` 是否与载荷匹配
3. 检查 `signedAt` 是否在合理的时间窗口内
4. 如果验证通过，分配对应的角色和权限

12.4.3 本地连接自动批准 vs 远程连接挑战签名

OpenClaw 的认证策略根据连接来源有不同的宽严度：

本地连接

当 `isLocalDirectRequest()` 返回 `true` 时（如 CLI 连接到本机的 Gateway），认证要求相对宽松：

- 本地 CLI 可以直接使用配置文件中的 Token
- 本地 Web 控制台可以通过 Origin 检查后免密访问
- 设备配对请求如果来自本机，可以自动批准

这是因为本地连接意味着攻击者需要已经获得了主机的访问权限，此时 Gateway 的认证已不是安全的主要防线。

远程连接

对于来自网络的远程连接，认证更加严格：

1. **设备必须已配对**：不能直接使用 Token（Token 可能在网络传输中泄露）
2. **使用挑战-响应签名**：Gateway 可以发送一个随机挑战（nonce），设备必须用私钥签名后返回
3. **Tailscale 验证**：如果通过 Tailscale VPN 连接，需要验证 Tailscale 用户身份

> **衍生解释**：挑战-响应认证（Challenge-Response Authentication）是一种经典的认证模式。服务器发送一个随机数（挑战），客户端用密钥对随机数进行加密或签名后返回（响应）。服务器验证响

> 应是否正确。这种方式的好处是密钥本身永远不会在网络上传输。NTLM、CRAM-MD5、以及 SSH 的公钥认证都使用了类似的机制。

Tailscale 认证

OpenClaw 对 Tailscale VPN 有特殊的集成支持。当 Gateway 以 Tailscale Serve 模式部署时，可以利用 Tailscale 的身份验证：

```
async function resolveVerifiedTailscaleUser(params: {
  req?: IncomingMessage;
  tailscaleWhois: TailscaleWhoisLookup;
}): Promise<{ ok: true; user: TailscaleUser } | { ok: false; reason: string }> {
  // 1. 检查 Tailscale 注入的用户头
  const tailscaleUser = getTailscaleUser(req);
  if (!tailscaleUser) return { ok: false, reason: "tailscale_user_missing" };

  // 2. 验证请求确实来自 Tailscale 代理
  if (!isTailscaleProxyRequest(req))
    return { ok: false, reason: "tailscale_proxy_missing" };

  // 3. 通过 Tailscale Whois API 验证用户身份
  const clientIp = resolveTailscaleClientIp(req);
  const whois = await tailscaleWhois(clientIp);

  // 4. 交叉验证：HTTP 头中的用户名必须与 Whois 结果一致
  if (normalizeLogin(whois.login) !== normalizeLogin(tailscaleUser.login))
    return { ok: false, reason: "tailscale_user_mismatch" };

  return { ok: true, user: { ... } };
}
```

> **衍生解释**：Tailscale 是一个基于 WireGuard 的零配置 VPN 服务。它的 Serve 功能允许将本地服务暴露到 Tailscale 网络中，并自动为每个请求注入用户身份信息（通过 HTTP 头）。OpenClaw 利用这个特性实现了"通过 VPN 身份认证 Gateway"——如果你在 Tailscale 网络中，你就不需要单独配置 Token。

验证流程的多重校验（检查 HTTP 头 → 验证代理身份 → Whois API 交叉验证）是为了防止**HTTP 头伪造**攻击——如果攻击者能够直接向 Gateway 发送请求并伪造 `Tailscale-User-Login` 头，没有 Whois 交叉验证的话就会绕过认证。

12.4.4 Origin 检查防止跨站 WebSocket 劫持

浏览器中的 WebSocket 连接面临一个特殊的安全威胁——**跨站 WebSocket 劫持**（Cross-Site WebSocket Hijacking, CSWSH）。

> **衍生解释**：跨站 WebSocket 劫持是 CSRF（跨站请求伪造）在 WebSocket 场景下的变种。当用户在浏览器中同时打开了恶意网页和 OpenClaw 控制台时，恶意网页可以尝试建立到 `ws://localhost:18789` 的 WebSocket 连接。由于浏览器会自动携带 cookies，如果 Gateway 不检查请求来源，恶意网页就可能成功建立连接并控制用户的 AI 助手。

Origin 检查逻辑

`src/gateway/origin-check.ts` 实现了 Origin 检查：

```typescript
export function checkBrowserOrigin(params: {
  requestHost?: string;      // 请求的 Host 头
  origin?: string;           // 请求的 Origin 头
  allowedOrigins?: string[]; // 配置的白名单
}): OriginCheckResult {
  // 1. Origin 头必须存在且有效
  const parsedOrigin = parseOrigin(params.origin);
  if (!parsedOrigin) {
    return { ok: false, reason: "origin missing or invalid" };
  }

  // 2. 检查白名单
  const allowlist = (params.allowedOrigins ?? [])
    .map((value) => value.trim().toLowerCase())
    .filter(Boolean);
  if (allowlist.includes(parsedOrigin.origin)) {
    return { ok: true };
  }

  // 3. 同源检查：Origin 的 host 必须与请求的 Host 头匹配
  const requestHost = normalizeHostHeader(params.requestHost);
  if (requestHost && parsedOrigin.host === requestHost) {
    return { ok: true };
  }

  // 4. 本地环回例外：localhost 到 localhost 始终允许
  const requestHostname = resolveHostName(requestHost);
  if (isLoopbackHost(parsedOrigin.hostname) && isLoopbackHost(requestHostname)) {
    return { ok: true };
  }

  return { ok: false, reason: "origin not allowed" };
}
```

检查逻辑遵循**最小特权原则**，按以下顺序判断：

1. **Origin 头必须存在**：浏览器在发起跨站请求时总是会附带 Origin 头。如果没有 Origin 头，可能是非浏览器客户端（不需要 Origin 检查）或者 Origin 被代理剥离了。
2. **白名单优先**：如果请求来源在配置的白名单中，直接通过。这允许管理员显式授权特定的外部来源。
3. **同源策略**：如果 Origin 的主机部分与请求的 Host 头匹配（即同一个域名），则允许。这是标准的同源策略。

4. **本地环回例外**：如果 Origin 和请求目标都是本地地址（localhost、127.0.0.1 等），则允许。这使得本地开发时 Web 控制台能正常工作，即使端口不同。

辅助函数

Origin 检查中用到的几个辅助函数处理了各种边界情况：

```
// 解析 Origin 字符串为结构化数据
function parseOrigin(originRaw?: string) {
  const trimmed = (originRaw ?? "").trim();
  if (!trimmed || trimmed === "null") return null; // "null" 是一个特殊值
  try {
    const url = new URL(trimmed);
    return {
      origin: url.origin.toLowerCase(),
      host: url.host.toLowerCase(),
      hostname: url.hostname.toLowerCase(),
    };
  } catch {
    return null;
  }
}

// 判断主机名是否为本地环回地址
function isLoopbackHost(hostname: string): boolean {
  return hostname === "localhost"
    || hostname === "::1"
    || hostname === "127.0.0.1"
    || hostname.startsWith("127.");
}
```

注意 `parseOrigin` 中对 `"null"` 字符串的处理——浏览器在某些情况下（如 `file://` 协议页面）会发送 `Origin: null`（字面量字符串），这不应该被当作有效的 Origin。

认证机制总结

用一张表格总结 OpenClaw 的多层认证机制：

认证方式	适用场景	安全级别	复杂度
Token	本地 CLI、受信服务端	中	低

认证方式	适用场景	安全级别	复杂度
Password	需密码保护的部署	中	低
设备配对	iOS/Android/macOS 应用	高	高
Tailscale	VPN 部署环境	高	中
Origin 检查	浏览器 WebSocket	补充	低
本地直连	本机连接	—	—

这些机制并不是互斥的，而是**分层叠加**的：

```
┌──────────────────────────────────┐
│     Origin 检查（浏览器层）        │ ← 防止 CSWSH
├──────────────────────────────────┤
│   Token / Password / 设备签名     │ ← 身份认证
├──────────────────────────────────┤
│   本地直连检测                     │ ← 环境感知
├──────────────────────────────────┤
│   Tailscale 身份验证               │ ← VPN 层认证
├──────────────────────────────────┤
│   TLS 加密（HTTPS/WSS）            │ ← 传输层安全
└──────────────────────────────────┘
```

> **v2026.3.9 Breaking Change：显式认证模式**
>
> 自 v2026.3.9 起，当 `gateway.auth.token` 和 `gateway.auth.password` **同时配置**时，必须显式设置 `gateway.auth.mode` 为 `"token"` 或 `"password"` 来指定优先使用哪种方式。之前版本会隐式选择其中一种，可能导致非预期行为。升级前请在 `openclaw.json` 中添加：
>
> ```
> { "gateway": { "auth": { "mode": "token" } } }
> ```

本节小结

OpenClaw 的认证系统设计体现了**深度防御**（Defense in Depth）的安全原则：

1. **多种认证方式**：Token、密码、设备配对、Tailscale——不同场景使用最合适的方式

2. **时间安全比较**：使用 `timingSafeEqual` 防止时序攻击
3. **设备配对的非对称密钥设计**：私钥永远不离开设备，通过签名验证身份
4. **重放攻击防护**：v2 协议引入 nonce，时间戳限制签名有效期
5. **Origin 检查**：针对浏览器环境的 CSWSH 防护
6. **本地连接优化**：对来自本机的请求提供便捷的认证路径

第 4 章总结

本章我们从协议设计哲学出发，逐层深入了 OpenClaw Gateway 的协议与类型系统：

- **4.1 协议设计哲学**：WebSocket 的选择理由、请求-响应-事件的混合帧模式、幂等键机制
- **4.2 TypeBox 类型模式**：单一定义多端复用的类型系统、JSON Schema 和 Swift 代码生成流水线
- **4.3 核心方法与事件**：80+ 个方法的功能分族、 `agent` / `send` / `sessions.*` 等核心操作、18 种事件类型
- **4.4 认证与授权**：Token 认证、设备配对、Tailscale 集成、Origin 检查的多层安全机制

这套协议和类型系统是 OpenClaw 所有上层功能（会话、Agent、工具、通道等）的基础通信层。在接下来的章节中，我们将基于这些协议方法，深入分析 Gateway 之上的各个子系统的实现。